

TECHCATALYST DE 2026

Python Fluency and Problem Solving

Turn small, visible steps into reliable transformation logic.

Week 3 Python

Read

Trace

Write

Test

Debug

Refactor

Python is a problem-solving workbench



What you will be able to do

Learning goals	By the end, you can...
Trace	Follow values through a small program before running it
Choose	Select list, tuple, dictionary, or set by behavior
Control	Use conditions and loops to express decisions clearly
Structure	Write focused functions with explicit inputs and outputs
Debug	Read the final traceback line and inspect the failing value
Transform	Use comprehensions when the data story stays readable
Test	Check a normal case and an edge case after each small step
Explain	Describe what the code changes and why

Weeks 1 to 4 are an AI-free zone

You cannot review code you could not have written.

What is allowed

- Write and debug your own code
- Use official docs and course references
- Ask peers to explain, not complete
- Call APIs and SDKs when the lesson requires them

Why we do this

- Fluency comes from typing and testing
- Error messages build debugging judgment
- Interviews still test fundamentals
- Later AI use should amplify skills you already own

01

Python fluency

Read it, write it, test it, explain it.

Fluency grows through a repeatable loop

Read: trace one value through the code and say what each line changes.

Write: solve one small step, run it, and inspect the result before adding more.

Debug: read the final traceback line first, then move upward to your code.

Choose collections by behavior

Container	Ordered	Mutable	Use it for
list	Yes	Yes	Sequences and repeated records
tuple	Yes	No	Fixed values and return groups
dict	Yes	Yes	Named fields and JSON-shaped records
set	No	Yes	Uniqueness and membership checks

The container communicates intent. Choose it before writing the loop.

Functions make transformation rules testable

```
def normalize_station(value: str) -> str:
    cleaned = value.strip().upper()
    if not cleaned:
        raise ValueError("station is required")
    return cleaned

station = normalize_station(" ma1bos ")
print(station) # MA1BOS
```

A strong function has one job, a descriptive name, explicit inputs, a predictable output, and a small surface for testing.

A traceback is a map, not a verdict

- Read the exception type and message
- Find the lowest frame from your file
- Inspect the values used on that line
- Change one assumption, then rerun

Start at the last line. Library internals matter only after your own inputs and assumptions are checked.

Traceback (most recent call last):

```
station = normalize_station(None)
```

```
AttributeError: NoneType has no attribute strip
```

Small drills build reusable moves

One drill should exercise one decision, then make the result visible.

Practice pattern

- Predict the output before running
- Write the smallest working version
- Test one normal and one edge case
- Explain the result to a partner

What success looks like

You can reproduce the move without copying, and you know where to look when it fails.

Comprehensions are compact loops, not code golf

```
rows = [  
    {"station": "MA1BOS", "tmax": 51},  
    {"station": "RI1PVD", "tmax": None},  
]  
  
valid = [row for row in rows if row["tmax"] is not None]  
lookup = {row["station"]: row["tmax"] for row in valid}  
print(lookup)
```

Use a comprehension when the read-select-transform story fits on one line. Use a regular loop when the logic needs branching, logging, or explanation.

Control flow makes decisions visible

Conditionals choose a path from evidence. Keep the condition explicit and test both outcomes.

Loops repeat one understandable step. Inspect the item and accumulated result while learning.

Functions name a rule, reduce repetition, and create a small surface for testing.

A small program is a data pipeline



- **Validate inputs at the boundary**
- **Transform one record or field at a time**
- **Make the output visible and testable**
- **Separate reading, transformation, and writing when possible**

Traceback detective challenge

Goal: diagnose three small failures using evidence instead of trial and error.

25 min · pairs

- Predict the intended output
- Read the exception type and final line
- Identify the failing value and violated assumption
- Make the smallest clear fix and rerun

Stretch: refactor one repeated cleaning rule into a function and test an edge case.

Deliverable: corrected code plus a short diagnosis for each failure.

Python official references

Reference	Use it for
Python tutorial	Language fundamentals and control flow
Python data structures	Lists, tuples, dictionaries, and sets
Python errors and exceptions	Tracebacks and exception handling
uv documentation	Reproducible daily Python projects
VS Code Python docs	Interpreter selection and debugging

Checked July 10, 2026. Prefer official documentation when older examples or remembered syntax disagree.

Fluency comes from visible reasoning

- 1 Trace one value through the code before adding complexity.
- 2 Choose collections by behavior, not habit.
- 3 Use functions to name and test transformation rules.
- 4 Treat tracebacks as evidence about a violated assumption.
- 5 Prefer readable loops and comprehensions over clever shortcuts.

Exit ticket: Which Python move can you now reproduce without copying?

Next: apply the same small-step reasoning to DataFrames and ETL logic.